

Wargames CTF 2024 Writeup

Team: Port_Knocking

Authors: Wong Wan Yin,



Unwanted Meow

Category: Forensic

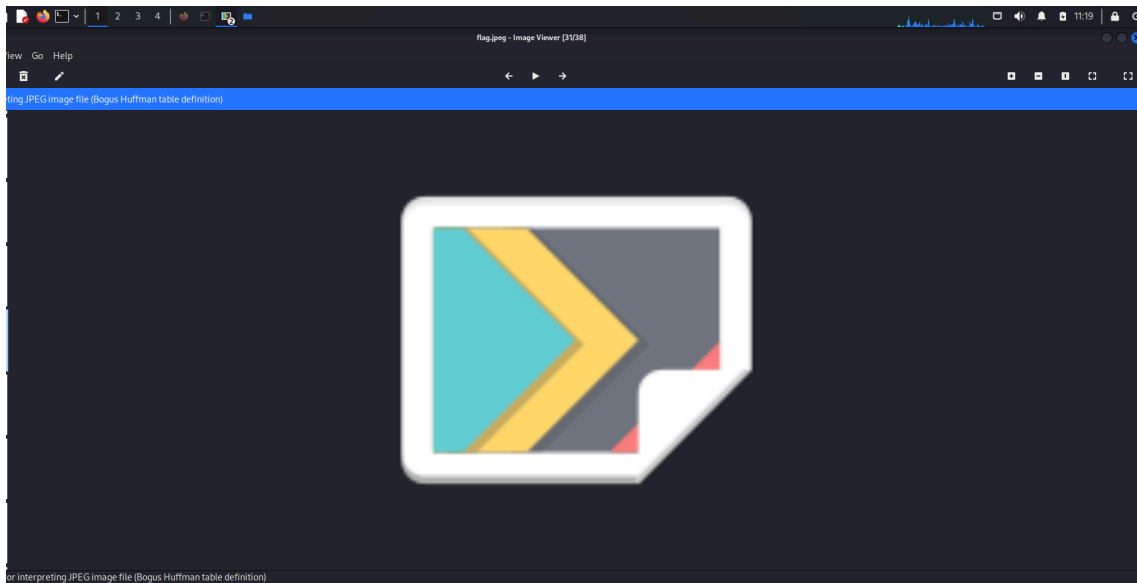
Description: Uh.. Oh.. Help me, I just browsing funny cats memes, when I click download cute cat picture, the file that been download seems little bit wierd. I accidently run the file making my files shredded. Ughh now I hate cat meowing at me.

I was given a file name **flag.shredded**.

First thing I did was to check what type of file **flag.shredded** is by using the command **file**.

```
(kali㉿kali)-[~/WargameCTF]
$ file flag.shredded
flag.shredded: JPEG image data, JFIF standard 1.01, density 25967×30465, segment length 16, thumbnail 1×44
```

Turns out, it is a JPEG file! Therefore, I tried changing the extension of the file to .jpg/.jpeg instead of .shredded. However, when I opened up the image, it wouldn't work. It seems like this jpeg file is corrupted.



Next, I used the command **exiftool** to view the details of **flag.shredded** to see if there is any valuable information. It turns out, there is!

```
(kali@kali)-[~/WargameCTF]
$ exiftool flag.shredded
ExifTool Version Number      : 12.76
File Name                    : flag.shredded
Directory                    : .
File Size                    : 35 kB
File Modification Date/Time   : 2024:12:28 11:13:44-05:00
File Access Date/Time        : 2024:12:28 11:21:32-05:00
File Inode Change Date/Time   : 2024:12:28 11:21:32-05:00
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
Warning                      : [minor] Skipped unknown 4 bytes after JPEG APP0 segment
JFIF Version                 : 1.01
Resolution Unit               : Unknown (109)
Thumbnail Width               : 1
Thumbnail Height              : 44
Exif Byte Order               : Big-endian (Motorola, MM)
Image Description              : funny cat looking shocked with mouth open portrait on green background with copy space
Orientation                   : Horizontal (normal)
X Resolution                  : 300
Y Resolution                  : 300
Current IPTC Digest           : 6498d85e387aa33cc8cda92f8f962bf2
By-line                       : Nils Jacobi
Caption-Abstract              : funny cat looking shocked with mouth open portrait on green background with copy space
Credit                       : Getty Images/iStockphoto
Image Width                   : 476
Image Height                  : 374
Encoding Process               : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components               : 3
Y Cb Cr Sub Sampling          : YCbCr4:2:0 (2 2)
Image Size                    : 476x374
Megapixels                    : 0.178
```

There is a warning indicating an issue with the **bytes** of the image, so I opened the file in a hex editor for further analysis.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	FF	D8	FF	E0	00	10	4A	46	49	46	00	01	01	6D	65	6F	ÿØÿà...JFIF...meow
00000010	77	01	01	2C	01	2C	00	00	FF	E1	00	AC	45	78	69	66	W.....ÿà.-Exif
00000020	00	00	4D	4D	00	2A	00	00	00	08	00	04	01	0E	00	02	..MM.*.....
00000030	00	00	00	56	00	00	00	3E	01	12	00	03	00	00	00	01	...V...>.....
00000040	00	01	00	00	01	1A	00	05	00	00	00	01	00	00	00	94	"
00000050	01	1B	00	05	00	00	00	01	00	00	00	9C	00	00	00	00	œ....
00000060	66	75	6E	6E	79	20	63	61	74	20	6C	6F	6F	6B	69	6E	funny cat lookin
00000070	67	20	73	68	6F	63	6B	65	64	20	77	69	74	68	20	6D	g shocked with m
00000080	6F	75	74	68	20	6F	70	65	6E	20	70	6F	72	74	72	61	outh open portra
00000090	69	74	20	6F	6E	20	67	72	65	65	6E	20	62	61	63	6B	it on green back
000000A0	67	72	6F	75	6E	64	20	77	69	74	68	20	63	6F	70	79	ground with copy
000000B0	20	73	70	61	63	65	00	00	01	2C	00	00	00	01	00	00	space...,.....
000000C0	01	2C	00	00	00	01	FF	E1	05	DF	68	74	74	70	3A	2F	.,....ÿà.\$http:/
000000D0	2F	6E	73	2E	61	64	6F	62	65	2E	63	6F	6D	2F	78	61	/ns.adobe.com/xa
000000E0	70	2F	31	2E	30	2F	00	3C	3F	78	70	61	63	6B	65	74	p/1.0/.<?xpacket
000000F0	20	62	65	67	69	6E	3D	22	EF	BB	BF	22	20	69	64	3D	begin="i»;" id=
00000100	22	57	35	4D	30	4D	70	43	65	68	69	48	7A	72	65	53	"W5M0MpCehiHzreS
00000110	7A	4E	54	63	7A	6B	63	39	64	22	3F	3E	0D	0A	3C	78	zNTczkc9d"?>..<x
00000120	3A	78	6D	70	6D	65	74	61	20	78	6D	6C	6E	73	3A	78	:xmpmeta xmlns:x
00000130	3D	22	61	6D	65	6F	77	64	6F	62	65	3A	6E	73	3A	6D	="ameowdobe:ns:m
00000140	65	74	61	6D	65	6F	77	2F	22	3E	0D	0A	09	3C	72	64	etameow/">...<rd
00000150	66	3A	52	44	46	20	78	6D	6C	6E	73	3A	72	64	66	3D	f:RDF xmlns:rdf=
00000160	22	68	74	74	70	3A	2F	2F	77	77	77	2E	77	33	2E	6F	"http://www.w3.o
00000170	72	67	2F	31	39	39	39	2F	30	32	2F	32	32	2D	72	64	rg/1999/02/22-rd
00000180	66	2D	73	79	6E	74	61	78	2D	6E	73	23	22	3E	0D	0A	f-syntax-ns#">..
00000190	09	09	3C	72	64	66	3A	44	65	73	63	6D	65	6F	77	72	..<rdf:Descmeowr
000001A0	69	70	74	69	6F	6E	20	72	64	66	3A	61	62	6F	75	74	ription rdf:about
000001B0	3D	22	22	20	78	6D	6C	6E	73	3A	70	68	6F	74	6F	73	=" " xmlns:photos
000001C0	68	6F	70	3D	22	68	74	74	70	3A	2F	2F	6E	73	2E	61	hop="http://ns.a
000001D0	64	6F	62	65	2E	63	6F	6D	2F	70	68	6F	74	6F	73	68	dobe.com/photosh
000001E0	6F	70	2F	31	2E	30	2F	22	20	78	6D	6C	6E	73	3A	49	op/1.0/" xmlns:I
000001F0	70	74	63	34	78	6D	70	43	6F	72	65	3D	22	68	74	74	ptc4xmpCore="htt

By analyzing the byte code, I noticed random instances of **'meow'** inserted into the code, which likely caused the file corruption. For example:

- adobe → ameowdobe
- meta → metameow
- Description → Descmeowription

To recover the corrupted file, I need to remove all instances of **'meow'** from the byte code. Therefore, I asked ChatGPT to generate a script that would automate this process.

```
def remove_meow(input_file, output_file):
    # Open the input file in binary mode
    with open(input_file, 'rb') as f:
        file_data = f.read()

    # Replace occurrences of 'meow' (ASCII 0x6D, 0x65, 0x6F, 0x77) with
    nothing (delete it)
    cleaned_data = file_data.replace(b'meow', b'')

    # Write the cleaned data to the output file
    with open(output_file, 'wb') as f:
```

```

        f.write(cleaned_data)

    print(f"Corrupted 'meow' data removed. Cleaned file saved as:
{output_file}")

# Specify the input and output files
input_file = 'flag.shredded' # Path to the corrupted file
output_file = 'flag_cleaned.jpg' # Path to save the cleaned file

# Run the function to remove 'meow'
remove_meow(input_file, output_file)

```

After running the script, a new file without any instances of **'meow'** will be generated as **flag_cleaned.jpg**. The flag is right there in the image!



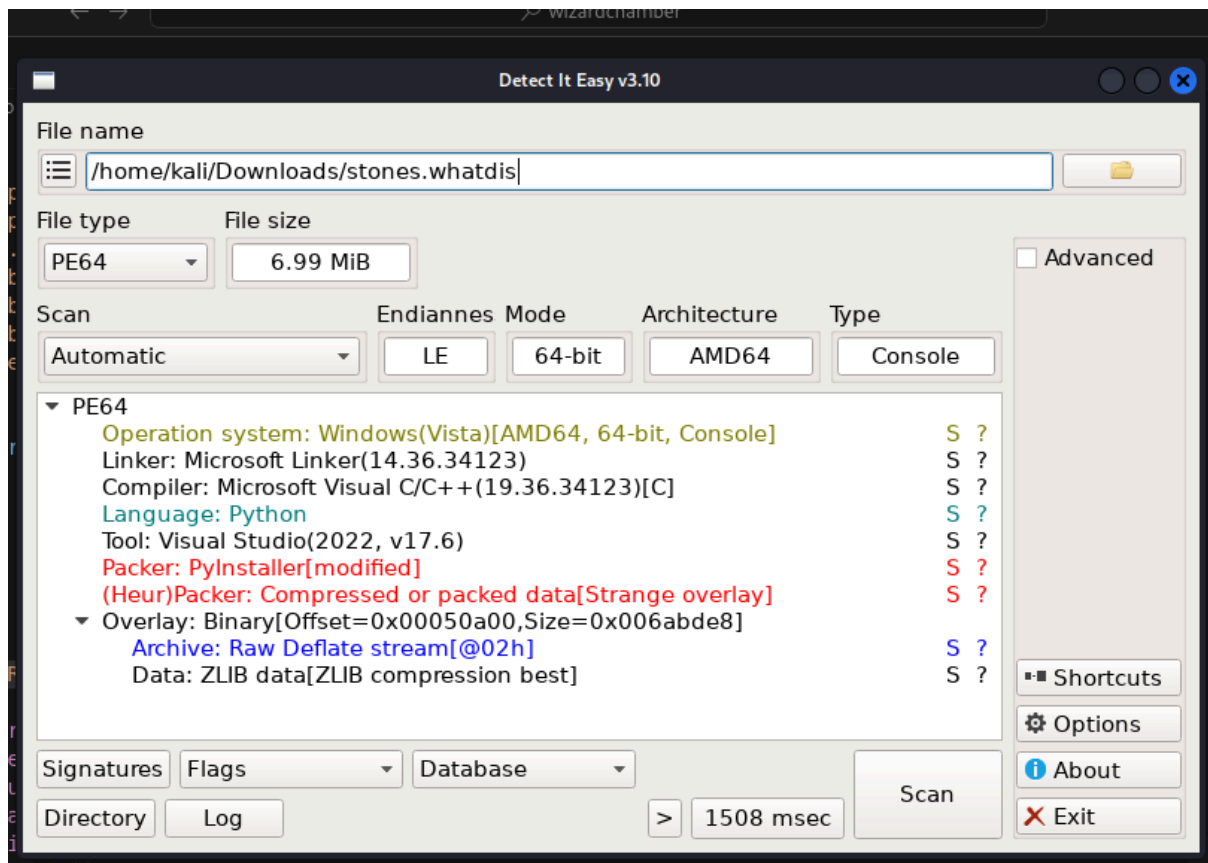
Flag: WGMY{4a4be40c96ac6314e91d93f38043a634}

Stones

Category: Reverse

Description: When Thanos snapped his fingers, half of the flag was blipped. We need the Avengers to retrieve the other half. There's no flag in the movie, but there is a slash flag on the server

Given the binary named “stones.whatdis”, with Detect-It-Easy we know that the executable is packed with PyInstaller and also a PE64 exe. This executable should be compiled into .pyc and then packaged into .exe.



Next we can use pyinstxtractor from <https://github.com/extremecoders-re/pyinstxtractor> to extract the decompiled python bytecode from the exe.

Here is how to extract it:

```
(kali@kali)~[~/Desktop/pyinstxtractor]
$ python3 pyinstxtractor.py ../Downloads/stones.exe
[+] Processing ../Downloads/stones.exe
[+] Pyinstaller version: 2.1+
[+] Python version: 3.10
[+] Length of package: 6995432 bytes
[+] Found 67 files in CArchive
[+] Beginning extraction... please standby
[+] Possible entry point: pyboot01_bootstrap.pyc
[+] Possible entry point: pyi_rth_inspect.pyc
[+] Possible entry point: CHAL-stones.pyc
[!] Warning: This script is running in a different Python version than the one used to build the executable.
[!] Please run this script in Python 3.10 to prevent extraction errors during unmarshalling
[!] Skipping pyz extraction
[+] Successfully extracted pyinstaller archive: ../Downloads/stones.exe
You can now use a python decompiler on the pyc files within the extracted directory
```

After the extraction, it will create an extracted directory within the tools folder. From there we were able to begin to analyze and reverse engineer the python decompiled code.

```
(kali@kali)-[~/Desktop/pyinstxtractor]
$ cd stones.exe_extracted

(kali@kali)-[~/Desktop/pyinstxtractor/stones.exe_extracted]
$ ls
api-ms-win-core-console-l1-1-0.dll      api-ms-win-core-sysinfo-l1-1-0.dll      _hashlib.pyd
api-ms-win-core-datetime-l1-1-0.dll     api-ms-win-core-timezone-l1-1-0.dll     libcrypto-1_1.dll
api-ms-win-core-debug-l1-1-0.dll        api-ms-win-core-util-l1-1-0.dll         libssl-1_1.dll
api-ms-win-core-errorhandling-l1-1-0.dll api-ms-win-crt-conio-l1-1-0.dll         _lzma.pyd
api-ms-win-core-file-l1-1-0.dll         api-ms-win-crt-convert-l1-1-0.dll       pyiboot01_bootstrap.pyc
api-ms-win-core-file-l1-2-0.dll         api-ms-win-crt-environment-l1-1-0.dll   pyimod01_archive.pyc
api-ms-win-core-file-l2-1-0.dll         api-ms-win-crt-file-system-l1-1-0.dll   pyimod02_importers.pyc
api-ms-win-core-handle-l1-1-0.dll       api-ms-win-crt-heap-l1-1-0.dll         pyimod03_ctypes.pyc
api-ms-win-core-heap-l1-1-0.dll         api-ms-win-crt-locale-l1-1-0.dll       pyimod04_pywin32.pyc
api-ms-win-core-interlocked-l1-1-0.dll  api-ms-win-crt-math-l1-1-0.dll        pyi_rth_inspect.pyc
api-ms-win-core-libraryloader-l1-1-0.dll api-ms-win-crt-process-l1-1-0.dll      python310.dll
api-ms-win-core-localization-l1-2-0.dll api-ms-win-crt-runtime-l1-1-0.dll      PYZ-00.pyz
api-ms-win-core-memory-l1-1-0.dll       api-ms-win-crt-stdio-l1-1-0.dll       PYZ-00.pyz_extracted
api-ms-win-core-namedpipe-l1-1-0.dll    api-ms-win-crt-string-l1-1-0.dll      _queue.pyd
api-ms-win-core-processenvironment-l1-1-0.dll api-ms-win-crt-time-l1-1-0.dll        select.pyd
api-ms-win-core-processthreads-l1-1-0.dll api-ms-win-crt-utility-l1-1-0.dll     _socket.pyd
api-ms-win-core-processthreads-l1-1-1.dll base_library.zip                      _ssl.pyd
api-ms-win-core-profile-l1-1-0.dll      _bz2.pyd                             struct.pyc
api-ms-win-core-rtlsupport-l1-1-0.dll  certifi                              ucrtdbase.dll
api-ms-win-core-string-l1-1-0.dll       CHAL-stones.pyc                     unicodedata.pyd
api-ms-win-core-synch-l1-1-0.dll        charset_normalizer                  VCRUNTIME140.dll
api-ms-win-core-synch-l1-2-0.dll        _decimal.pyd
```

From here, we can know the exe file is from a python code (.py) is then compiled into python compiled bytecode (.pyc). That is the reason PyInstaller appears in Detect-It-Easy as it bundles all these .pyc files with python interpreter which if we can be found in base_library.zip along with its dependencies. The important files that we should notice is CHAL-stones.pyc which will be the main script. The rest of the files are compressed modules and Windows API for the exe to run without requiring Python as a native environment.

The next focus is to decompile CHAL-stones.pyc, we can utilize pylingual <https://pylingual.io/>.

```
binding * CHAL-stones.pyc Python3.10

Python Code - Decompilation Success

1 # Source timestamp: 1970-01-01 00:00:00 UTC (0)
2
3 import requests
4 # Source timestamp: 1970-01-01 00:00:00 UTC (0)
5
6 import requests
7 from datetime import datetime
8 from urllib.request import urlopen
9 from datetime import datetime
10 server_url = 'http://3.142.133.106:8000/'
11 current_time = urlopen('http://just-the-time.appspot.com/')
12 current_time = current_time.read().strip()
13 current_time = current_time.decode('utf-8')
14 current_date = current_time.split(' ')[0]
15 local_date = datetime.now().strftime('%Y-%m-%d')
16 if current_date == local_date:
17     print("We're gonna need a really big brain; bigger than his?")
18 first_flag = 'WGM{1d2993}'
19 user_date = current_date

Submitted Bytecode

8 76 STORE_NAME 5 (current time)
9 78 LOAD_NAME 5 (current time)
10 80 LOAD_METHOD 9 (split)
11 82 LOAD_CONST 7 (" ")
```

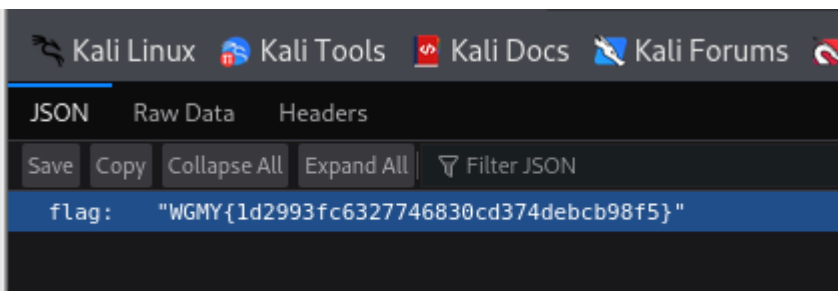
```

import requests
from datetime import datetime
from urllib.request import urlopen
from datetime import datetime
server_url = 'http://3.142.133.106:8000/'
current_time = urlopen('http://just-the-time.appspot.com/')
current_time = current_time.read().strip()
current_time = current_time.decode('utf-8')
current_date = current_time.split(' ')[0]
local_date = datetime.now().strftime('%Y-%m-%d')
if current_date == local_date:
    print("We're gonna need a really big brain; bigger than his?")
first_flag = 'WGMY{1d2993'
user_date = current_date
params = {'first_flag': first_flag, 'date': user_date}
response = requests.get(server_url, params=params)
if response.status_code == 200:
    print(response.json()['flag'])
else:
    print(response.json()['error'])

```

Based on the challenge python code, it involves a server flag checker where it get the current time from an external time server (<http://just-the-time.appspot.com/>) and compares it with the local system date. If the dates match, it proceeds to send a GET request to a server (<http://3.142.133.106:8000/>) with two parameters: `first_flag` (a partial flag, `WGMY{1d2993`) and the current date. The server's response is expected to contain the complete flag in JSON format, which is printed if the request succeeds. Otherwise, an error message is printed.

Based on the hint given from the description, there is a “slash” flag (<http://3.142.133.106:8000/flag>), it redirects to a youtube video <https://www.youtube.com/watch?v=V0zJb2K4Yi8&t=75s> along with the json object name of “upload date”, so the upload date of the video is Jul 25, 2022. So, we have to craft the parameter with the upload date:
http://3.142.133.106:8000/?first_flag=WGMY{1d2993&date=2022-07-25



Flag: WGMY{1d2993fc6327746830cd374debc98f5}

Sudoku

Category: Reverse

Description: Easy stuff, frfr. You dont need to brute force or guess anything. The final flag don't have any dot (.)

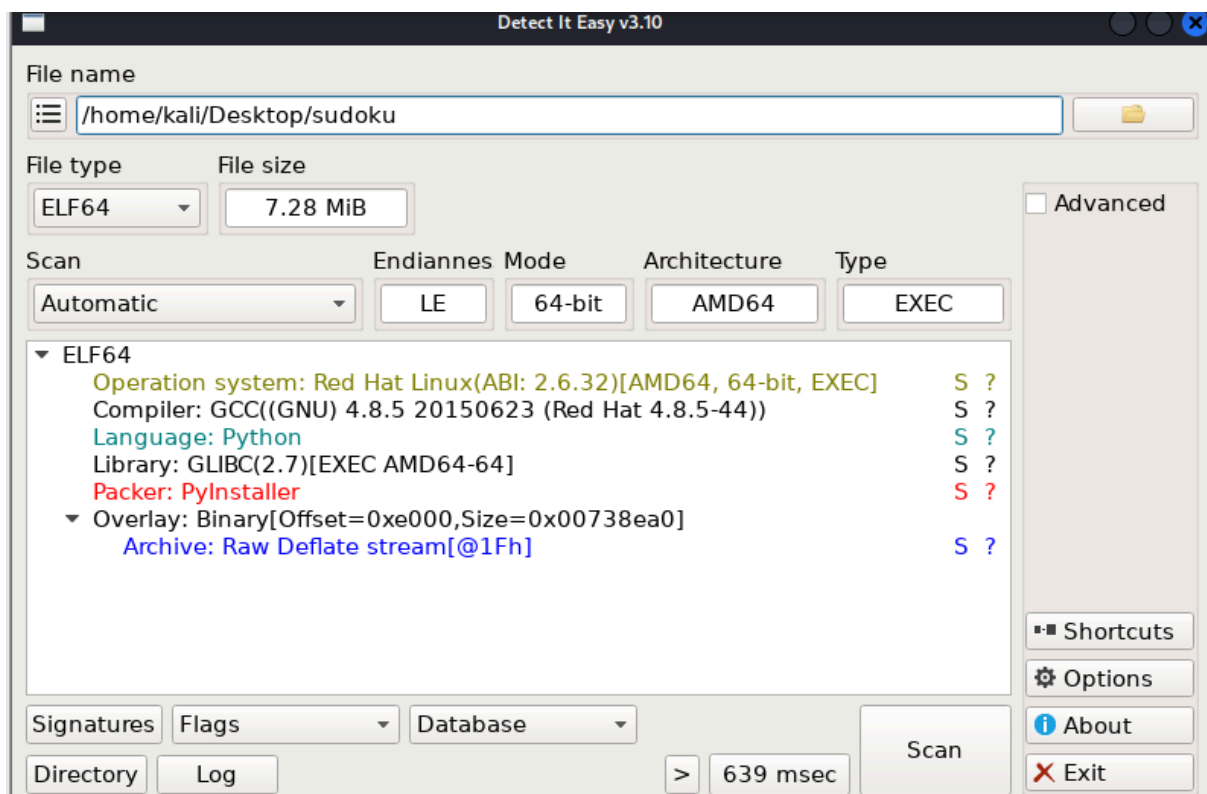
Given the sudoku.zip and unzip we got an ELF executable and out.enc

```
$ file sudoku
sudoku: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=979c889cc9ca56a33ed67aeb4d2d05cb8db2df1b, for GNU/Linux
2.6.32, stripped
```

and check the contents of out.enc

```
$ cat out.enc
z v7o1 an7570 9d.tl3 7.4b 7n2pws .qodx v7oc ye68u m.7r,
t728{09er1bzb9sx5sosu7719besr39zscbx}
```

Next, use Detect-It-Easy to verify the binary executable.



As it shows the binary is packed with PyInstaller which is Python (.py) file converted into an executable. Next, use pyinstxtractor from

<https://github.com/extremecoders-re/pyinstxtractor> to extract the decompiled python bytecode from the binary.

To extract the .pyc from the binary

```
(kali㉿kali)-[~/Desktop/pyinstxtractor]
$ python3 pyinstxtractor.py ../../Downloads/sudoku
[+] Processing ../../Downloads/sudoku
[+] Pyinstaller version: 2.1+
[+] Python version: 3.11
[+] Length of package: 7574432 bytes
[+] Found 31 files in CArchive
[+] Beginning extraction... please standby
[+] Possible entry point: pyiboot01_bootstrap.pyc
[+] Possible entry point: pyi_rth_inspect.pyc
[+] Possible entry point: sudoku.pyc
[!] Warning: This script is running in a different Python version than the one used to build the executable.
[!] Please run this script in Python 3.11 to prevent extraction errors during unmarshalling
[!] Skipping pyz extraction
[+] Successfully extracted pyinstaller archive: ../../Downloads/sudoku
```

After extraction, head to the directory where it stored all compiled .pyc file

```
(kali㉿kali)-[~/Desktop/pyinstxtractor/sudoku_extracted]
$ ls
abc.pyc          keyword.pyc      operator.pyc     sre_compile.pyc
base_library.zip libbz2.so.1.0   os.pyc          sre_constants.pyc
codecs.pyc       libcrypto.so.3  posixpath.pyc   sre_parse.pyc
collections      lib-dynload     pyiboot01_bootstrap.pyc stat.pyc
_collections_abc.pyc libexpat.so.1  pyimod01_archive.pyc struct.pyc
copyreg.pyc      liblzma.so.5   pyimod02_importers.pyc sudoku.pyc
encodings        libpython3.11.so.1.0 pyimod03_ctypes.pyc traceback.pyc
enum.pyc         libz.so.1       pyi_rth_inspect.pyc types.pyc
functools.pyc    libzstd.so.1    PYZ-00.pyz      warnings.pyc
genericpath.pyc  linecache.pyc   PYZ-00.pyz_extracted weakref.pyc
heapq.pyc        locale.pyc      re              _weakrefset.pyc
io.pyc           ntpath.pyc     reprlib.pyc
```

Next use pylingual <https://pylingual.io/> to decompile the sudoku.pyc.

```
Python Code - Decompile Success

4 # Source timestamp: 1970-01-01 00:00:00 UTC (0)
5
6 import random
7 alphabet = 'abcdefghijklmnopqrstuvwxyz1234567890.'
8 plaintext = 't.e qu.c. brown .ox .umps over t.e lazy do., w.my{[REDACTED]}'
9
10 def makeKey(alphabet):
11     alphabet = list(alphabet)
12     random.shuffle(alphabet)
13     return ''.join(alphabet)
14 key = makeKey(alphabet)
15
16 def encrypt(plaintext, key, alphabet):
17     keyMap = dict(zip(alphabet, key))
18     return ''.join((keyMap.get(c.lower(), c) for c in plaintext))
19 enc = encrypt(plaintext, key, alphabet)

Submitted Bytecode

1 0 LOAD_CONST 0 (0)
```

Here is the python code:

```
import random
alphabet = 'abcdefghijklmnopqrstuvwxyz1234567890.'
plaintext = 't.e qu.c. brown .ox .umps over t.e lazy do.,
w.my{[REDACTED]}'

def makeKey(alphabet):
    alphabet = list(alphabet)
    random.shuffle(alphabet)
    return ''.join(alphabet)
key = makeKey(alphabet)

def encrypt(plaintext, key, alphabet):
    keyMap = dict(zip(alphabet, key))
    return ''.join((keyMap.get(c.lower(), c) for c in plaintext))
enc = encrypt(plaintext, key, alphabet)
```

From this script is that it implements a function to substitution encryption with a random key. Since we have the **out.enc** file, we can just reverse it

Here is the reverse script:

```
alphabet = 'abcdefghijklmnopqrstuvwxyz1234567890.'
encrypted = "z v7o1 an7570 9d.tl3 7.4b 7n2pws .qodx v7oc ye68u
m.7r, t728{09er1bzbs9sx5sosu7719besr39zscbx}"
known_plaintext = '0 t.e1 qu.c.2 brown3 .ox4 .umps5 over6 t.e7
lazy8 do.9, w.my'
partial_key_map = {}
for p_char, e_char in zip(known_plaintext, encrypted):
    if p_char in alphabet:
        partial_key_map[p_char] = e_char
key = ''.join(partial_key_map.get(char, '?') for char in alphabet)
def decrypt(encrypted, key, alphabet):
    reverse_key_map = dict(zip(key, alphabet))
    return ''.join((reverse_key_map.get(c, c) for c in
encrypted))

decrypted = decrypt(encrypted, key, alphabet)
print(f"Partial Decrypted Text: {decrypted}")
```

After running the script, we will get this:

```
$ python3 solve.py  
Partial Decrypted Text: 0 t.e1 qu.c.2 brown3 .ox4 .umps5 over6  
t.e7 lazy8 do.9, w.my{2ba914045b56c5e58..1b4a593b05746}
```

Ok now, we got this partial flag, but if we look at the alphabets, notice there are missing characters.

```
alphabet = 'abcdefghijklmnopqrstuvwxyz1234567890.'  
#missing letter 'fghijk' -> this makes the dots in the out.enc
```

And also MD5 hash, which is part of the flag, it consist of hexadecimal characters, the only hexadecimal characters missing from the “alphabets” is f. So, now can we construct the full form flag:

Flag: wgmy{2ba914045b56c5e58ff1b4a593b05746}

Oh Man!

Category: Forensic

Description: We received a PCAP file from an admin who suspects an attacker exfiltrated sensitive data. Can you analyze the PCAP file and uncover what was stolen?

Disclaimer: Our team was unable to solve this challenge during the CTF, but I (Wan Yin) made an attempt to solve it afterwards, with guidance from another team's write-up (by [k.eii](#)). This write-up reflects my understanding of the process.

I am given a PCAP file to analyse so let's open up **Wireshark**!

The first thing I do whenever I open a pcap file is to observe what protocols are present. In this pcap file, there are SMB, SMB2, TCP, **DCERPC**, **IWBEMSERVICES**, **ISystemActivator**, **IRemUnknown** and **IWBEMLEVEL1LOGIN**.

*The ones being bold are new protocols for me 😊

I started filtering the protocols one by one to have an overview of it and I realise SMB2 protocols are encrypted so this gives me a heads up to look for ways to decrypt it.

Next, I start researching about the **NEW** protocols and what they do?

1. DCERPC (Distributed Computing Environment Remote Procedure Call)

- **Purpose:** Allows remote execution of procedures on another machine. Often used in Windows environments for administrative operations.

2. ISystemActivator

- **Purpose:** Part of DCOM (Distributed Component Object Model). Used to activate COM objects on remote machines.

3. IWBEMLEVEL1LOGIN

- **Purpose:** Related to WBEM (Web-Based Enterprise Management). Handles authentication and initialization for WBEM operations.

4. IWbemServices

- **Purpose:** Used to perform management tasks such as querying WMI (Windows Management Instrumentation) services.

5. IRemUnknown

- **Purpose:** A DCOM interface used for managing remote references to COM objects.

Now, back to the challenge. If we want to determine what data is being exfiltrated by the attacker, we can filter for **IWbemServices** traffic. This is because **IWbemServices** is involved in querying and executing commands on Windows Management Instrumentation (WMI), which can be exploited to interact with system data. By analyzing this traffic, we can detect potential attacker activities, such as retrieving sensitive information or issuing commands to exfiltrate data, helping us track the exfiltration process.

No.	Time	Source	Destination	Protocol	Length	Info
159	0.354284	192.168.68.190	192.168.68.209	IWbemServices	226	GetObject request Object=win32_Process
168	0.393349	192.168.68.209	192.168.68.190	IWbemServices	806	GetObject response[Malformed Packet]
171	0.464562	192.168.68.190	192.168.68.209	IWbemServices	726	ExecMethod request[Long frame (1956 bytes)]
173	0.493829	192.168.68.209	192.168.68.190	IWbemServices	1278	ExecMethod response, Error: Unknown DOS error 0x00020000[Long frame (1148 bytes)]
258	2.791693	192.168.68.190	192.168.68.209	IWbemServices	226	GetObject request Object=win32_Process
267	2.814099	192.168.68.209	192.168.68.190	IWbemServices	806	GetObject response[Malformed Packet]
272	2.983551	192.168.68.190	192.168.68.209	IWbemServices	798	ExecMethod request[Long frame (2028 bytes)]
274	2.986720	192.168.68.209	192.168.68.190	IWbemServices	1278	ExecMethod response, Error: Unknown DOS error 0x00020000[Long frame (1148 bytes)]

Let's focus on the **ExecMethod** packets as it is where commands are likely executed and might include activities of exfiltration of data.

Frame 171: 726 bytes on wire (5808 bits), 726 bytes captured (5808 bits) on interface
> Ethernet II, Src: VMware_f2:36:ad (00:0c:29:f2:36:ad), Dst: VMware_5b:92:7f (00:0c:29:5b:92:7f)
> Internet Protocol Version 4, Src: 192.168.68.190, Dst: 192.168.68.209
> Transmission Control Protocol, Src Port: 39572, Dst Port: 49758, Seq: 3549, Ack: 23343
> [2 Reassembled TCP Segments (2132 bytes): #170(1460), #171(672)]
> Distributed Computing Environment / Remote Procedure Call (DCE/RPC) Request, Fragment:
> IWbemServices (pid1), ExecMethod

01d0 01 dc 00 00 00 00 5f 5f 50 41 52 41 4d 45 54 45 PARAMETER
01e0 52 53 00 01 63 00 6d 00 64 00 2e 00 65 00 78 00 RS-c-m d..e.x
01f0 65 00 20 00 2f 00 51 00 20 00 2f 00 63 00 20 00 e- /Q- /c-
0200 74 00 61 00 73 00 6b 00 6c 00 60 00 73 00 74 00 b-a-s-k- i-i-s-t-
0210 20 00 2f 00 76 00 20 00 2f 00 66 00 6f 00 20 00 /v- /f-
0220 63 00 73 00 76 00 20 00 7c 00 20 00 66 00 69 00 c-s-v- /-f-i-
0230 6a 00 64 00 73 00 74 00 72 00 20 00 2f 00 69 00 n-d-s-t- n- /i-
0240 20 00 22 00 6c 00 73 00 61 00 73 00 73 00 22 00 ..l-s- a-s-"
0250 20 00 31 00 3e 00 20 00 5c 00 5c 00 31 00 32 00 .1- > \-1-2-
0260 37 00 2e 00 30 00 2e 00 30 00 2e 00 31 00 5c 00 7- .0- .0-1-
0270 43 00 24 00 5c 00 57 00 69 00 6e 00 64 00 6f 00 C-\$- \W- i-n-d-o-
0280 77 00 73 00 5c 00 54 00 65 00 6d 00 70 00 5c 00 w-s- \T- e-m-p- \-
0290 77 00 71 00 70 00 69 00 5a 00 6f 00 20 00 32 00 w-g-p-i- z-o- 2-
02a0 3e 00 26 00 31 00 00 00 01 43 00 3a 00 5c 00 00 >-8-1-... C- \-
02b0 00 aa 5b 8e 00 00 00 00 00 00 00 00 00 0a 05 ..[.....
02c0 00 00 81 35 01 00 01 00 00 39 02 15 96 cd a5 ...5....-g.....

When investigating the ExecMethod requests, I saw something in the byte strings that looks like a command, so let's follow the TCP stream of those packets to explore further.

```
.....)....
.....f.....D....ID.
.....6....
.....f.....D.....object:Win32_ProcessStartup.....
.....
.....
.....
.....0.....PARAMETERS..c.m.d...e.x.e. ./Q. ./c. .t.a.s.k.l.i.s.t. ./
v. ./f.o. .c.s.v. .|. .f.i.n.d.s.t.r. ./i. ."l.s.a.s.s.". .1.>. .\.\127.0.0.1\.\C$. \W.i.n.d.o.w.s.
\T.e.m.p.\w.q.p.i.Z.o. .2.>.&.1...C:.\....[.....
.....5.....9.....
.....
.....e...e...MEOW.....s...M...K...$...E:.....K...$...5...xV4.-.....
.....*...4.....__PARAMETERS..abstract.....Pro
cessId..uint32.....
.....5.....Out.....
.....5...\.....Win32API|Process and Thread Functions|CreateProcess|lpProcessInformation|dwProce
ssId..MappingStrings.....)....
.....5...\.....ID.....6...
.....X...\.....0.....uint32.....ReturnValue..uint32.....
.....
.....uint32..out.....
```

In the TCP stream, we can clearly see the command:

```
cmd.exe /Q /c tasklist /v /fo csv | findstr /i "lsass" 1>
\\127.0.0.1\C$\Windows\Temp\wqpiZo 2>&1
```

Explanation of command

1. Executing a Tasklist:
 - The attacker enumerated running processes to identify **lsass**, likely targeting it to extract sensitive data like credentials or hashes.
2. Saving the Results:
 - The results were saved in a temporary file **wqpiZo** in the **Windows Temp directory**.

Purpose of the Activity:

- The attacker may be preparing to:
 - Exfiltrate the **wqpiZo** file containing process details.
 - Target **lsass** for credential dumping using tools like **Mimikatz**.

What is lsass?

The Local Security Authority Subsystem Service (LSASS) is a critical Windows process that manages user authentication, security policies, and auditing:

- User authentication: LSASS verifies users logging on to a Windows computer or server.

- Security policies: LSASS enforces security policies on the system.
- Auditing: LSASS writes to the Windows Security Log.
- Active Directory: On an Active Directory domain controller, LSASS provides Active Directory database lookups, authentication, and replication.

Here is another command from another packet:

```
.....In.
.....
.....L....WMI|Win32_ProcessStartup..MappingStrings.
.....)....
.....f.....D....ID.
.....6...
.....f.....D.....object:Win32_ProcessStartup.....
.....
.....C.....0.....$.PARAMETERS..c.m.d...e.x.e. ./Q. ./c. C:.\W.i.n.d.o.w.s.
\T.e.m.p.\n.a.n.o...e.x.e. --pid. 840. --w.r.i.t.e. C:.\W.i.n.d.o.w.s.\T.e.m.p.\2.0.2.4.1.2.2.
5._1.9.3.9...l.o.g. 1>. \1.2.7...0...1.\C$. \W.i.n.d.o.w.s.\T.e.m.p.\R.x.H.m.E.j. 2>.&1...C.:
.\.....
.....5.....p.Q....d....
.....e...MEOW.....s...M...K.$...E:.....K.$...5...xV4.-.....
..*...4....._PARAMETERS..abstract.....Pro
cessId..uint32.....
.....5....Out.....
.....5...\Win32API|Process and Thread Functions|CreateProcess|lpProcessInformation|dwProce
ssId..MappingStrings.....)....
.....5...\ID.....6...
.....X...\0.....uint32.....ReturnValue..uint32.....
..
.....uint32..out.....
```

```
cmd.exe /Q /c C:\Windows\Temp\nano.exe --pid 840 --write
C:\Windows\Temp\20241225_1939.log 1> \127.0.0.1\C$\Windows\Temp\RxHmEj
2>&1
```

Explanation of of command

1. Dumping Data:
 - The **nano.exe** tool appears to interact with **PID 840** (possibly **lsass.exe**) and dumps data into a log file **20241225_1939.log**.
2. Exfiltration Preparation:
 - The log file is saved locally in the **Temp** directory and its output/error is redirected to another file (**RxHmEj**), which could be exfiltrated later.

Now that we know how the data is being exfiltrated and where it is stored, we just have to somehow access or retrieve the file to find the flag. But how?

By decrypting the **SMB2** packets!

This is because the exfiltrated data is likely encrypted within these packets, and **SMB2** is commonly used for file sharing and communication. If the attacker encrypts the data before transmission, decrypting the SMB2 traffic will allow us to access the

original file. Once decrypted, we can analyze the file to locate the flag or other relevant information.

I then started to research on how to **decrypt SMB2** in Wireshark and I found a writeup similar to what I was looking for!

<https://medium.com/maverislabs/decrypting-smb3-traffic-with-just-a-pcap-absolutely-maybe-712ed23ff6a2>

I followed the writeup to get the **password** and the **Random Session Key** to decrypt the SMB2 protocols.

hash.txt

```
Administrator::DESKTOP-PMNU0JK:7aaff6ea26301fc3:ae62a57caaa5dd94b68def8f
b1c192f3:010100000000000008675779b2e57db01376f686e57504d7700000000200
1e004400450053004b0054004f0050002d0050004d004e00550030004a004b000100
1e004400450053004b0054004f0050002d0050004d004e00550030004a004b000400
1e004400450053004b0054004f0050002d0050004d004e00550030004a004b000300
1e004400450053004b0054004f0050002d0050004d004e00550030004a004b000700
08008675779b2e57db010900280063006900660073002f004400450053004b005400
4f0050002d0050004d004e00550030004a004b00000000000000000000000000
```

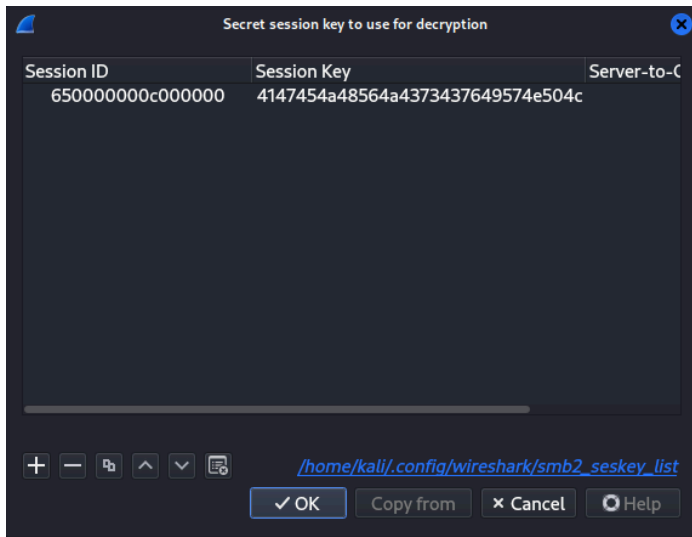
Command: **hashcat -a 0 -m 5600 hash.txt /usr/share/wordlists/rockyou.txt**

Password: **password<3**

Session key: **4147454a48564a4373437649574e504c** (I got this by using the python script in the writeup above)

Session id: **650000000c000000**

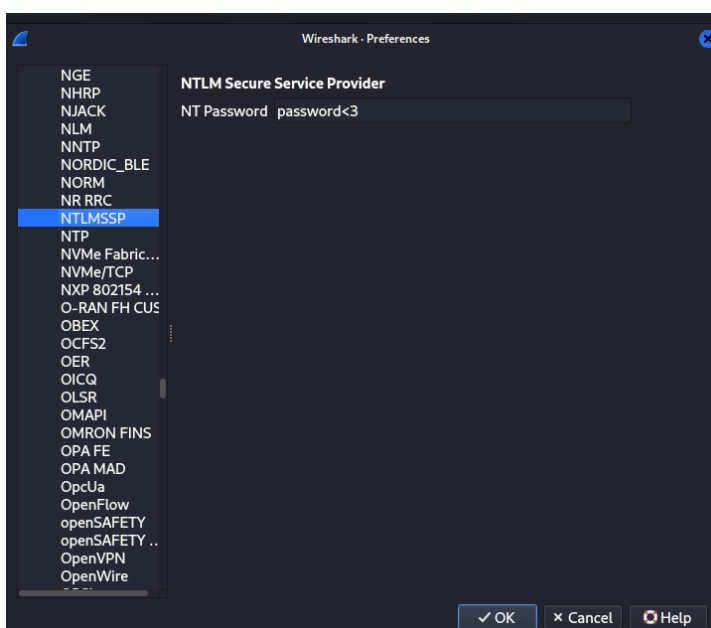
However, when I entered the session ID and session key for decryption, I thought it will successfully be decrypt but NOTHING happened! This is where I got lost and stuck for a VERY LONG TIME and eventually gave up 😞



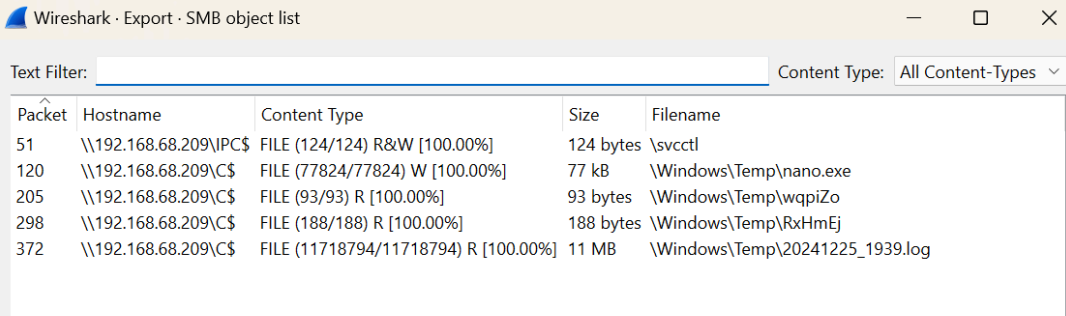
After the CTF, I read about the writeup for this challenge from another team and realised that before decrypting SMB2, I have to enter the password for **NTLMSSP**.

This is because **NTLMSSP** (NT LAN Manager Security Support Provider) is responsible for the **authentication process in SMB2**. The password is a critical part of generating the session key used to decrypt SMB2 packets. Without providing the correct password, the session key cannot be derived, and the encrypted SMB2 traffic remains inaccessible.

The step I missed was inputting the **NTLMSSP password**, which is essential for completing the authentication process and successfully decrypting the data. Understanding this gap made me realize the importance of fully analyzing authentication protocols when dealing with encrypted traffic.



Finally, after entering the password, SMB2 has been successfully decrypted. Now we can go to **File > Export Objects > SMB** to export the files that we have previously seen in the command executed by the attacker.



The image shows a Wireshark window titled "Wireshark · Export · SMB object list". It has a "Text Filter:" field and a "Content Type:" dropdown set to "All Content-Types". Below is a table with 5 columns: Packet, Hostname, Content Type, Size, and Filename.

Packet	Hostname	Content Type	Size	Filename
51	\\192.168.68.209\IPC\$	FILE (124/124) R&W [100.00%]	124 bytes	\svcctl
120	\\192.168.68.209\C\$	FILE (77824/77824) W [100.00%]	77 kB	\Windows\Temp\nano.exe
205	\\192.168.68.209\C\$	FILE (93/93) R [100.00%]	93 bytes	\Windows\Temp\wqpiZo
298	\\192.168.68.209\C\$	FILE (188/188) R [100.00%]	188 bytes	\Windows\Temp\RxHmEj
372	\\192.168.68.209\C\$	FILE (11718794/11718794) R [100.00%]	11 MB	\Windows\Temp\20241225_1939.log

After exporting the files, I focused on the content of **20241225_1939.log**. This is because, earlier, the attacker appeared to have dumped sensitive data, likely from **lsass**, into this log file. However, the file couldn't be opened in any text editor, suggesting it might be corrupted or intentionally manipulated to appear broken.

Next, I begin exploring the other files that have been exported. I start with examining what file types they are by using the **file** command again.

```
(kali㉿kali)-[~/WargameCTF/ohman]
$ file %5csvcctl
%5csvcctl: Targa image data - RGB 1 x 65536 x 24 +48 ""

(kali㉿kali)-[~/WargameCTF/ohman]
$ file %5cWindows%5cTemp%5cnano.exe
%5cWindows%5cTemp%5cnano.exe: PE32+ executable (console) x86-64 (stripped to external PDB), for MS Windows, 10 sections

(kali㉿kali)-[~/WargameCTF/ohman]
$ file %5cWindows%5cTemp%5cRxHmEj
%5cWindows%5cTemp%5cRxHmEj: ASCII text, with CRLF line terminators

(kali㉿kali)-[~/WargameCTF/ohman]
$ file %5cWindows%5cTemp%5cwqpiZo
%5cWindows%5cTemp%5cwqpiZo: ASCII text, with CRLF line terminators

(kali㉿kali)-[~/WargameCTF/ohman]
$
```

As we can see file **RxHmEj** and **wqpiZo** are ascii files, so I continue to **cat** out the text.

```
(kali㉿kali)-[~/WargameCTF/ohman]
$ cat %5cWindows%5cTemp%5cRxHmEj
The minidump has an invalid signature, restore it running:
scripts/restore_signature 20241225_1939.log
Done, to get the secretz run:
python3 -m pypykatz lsa minidump 20241225_1939.log

(kali㉿kali)-[~/WargameCTF/ohman]
$ cat %5cWindows%5cTemp%5cwqpiZo
"lsass.exe","840","Services","0","24,332 K","Unknown","NT AUTHORITY\SYSTEM","0:00:00","N/A"
```

File **wqpiZo** hinted that the context of the log file might be from **lsass.exe** which contains sensitive information such as credentials, security tokens, and other authentication data.

File **RxHmEj** hinted that the minidump (20241225_1939.log) has an invalid signature and if we want to restore it we can run a **script** called **restore_signature**. After searching online, I found that we can use the script by git cloning this GitHub <https://github.com/fortra/nanodump>

After running the script to restore the signature, the output prompts us to execute another command to analyze the dump. Let's proceed with that.

```
(kali㉿kali)-[~/WargameCTF/ohman/nanodump/scripts]
$ ./restore_signature ../../%5cWindows%5cTemp%5c20241225_1939.log
done, to analyze the dump run:
python3 -m pypykatz lsa minidump ../../%5cWindows%5cTemp%5c20241225_1939.log

(kali㉿kali)-[~/WargameCTF/ohman/nanodump/scripts]
$ python3 -m pypykatz lsa minidump ../../%5cWindows%5cTemp%5c20241225_1939.log

= WDIGEST [52d2a]=
  username Administrator
  domainname DESKTOP-PMNU0JK
  password None
  password (hex)

= Kerberos =
  Username: Administrator
  Domain: DESKTOP-PMNU0JK

= WDIGEST [52d2a]=
  username Administrator
  domainname DESKTOP-PMNU0JK
  password None
  password (hex)

= CREDMAN [52d2a]=
  luid 339242
  username wgmy
  domain wargames.my
  password wgmy{fbba48bee397414246f864fe4d2925e4}
  password (hex) 770067006d0079007b00660062006200610034003800620065006500330039
003700340031003400320034003600660038003600340066006500340064003200390032003500650034007d0000
000000

= DPAPI [52d2a]=
  luid 339242
  key_guid 3f2e1f8e-6e46-401f-9eaf-c04ae5fce736
  masterkey b44f25f6d196a92f77f22ecc14db19b574b3f266b44a48ed132b8268d3241a966b
15d937cbfc6b6c364222743fd93b3f0ecb1c6c4ebe326727f981376c34c7f0
  sha1_masterkey adc7c99f9546f4374b9ee78d6a56fea568cea802

= LogonSession =
  authentication_id 997 (3e5)
  session_id 0
  username LOCAL SERVICE
  domainname NT AUTHORITY
  logon_server
  logon_time 2024-12-26T00:08:18.879757+00:00
```

Finally! There's the flag! **wgmy{fbba48bee397414246f864fe4d2925e4}**